



HASHING

Hashing hota kya h?

Hashing ek **efficient searching technique** hai jo kisi value ko ek hash function me pass krke uska key nikalti h and us jagah par us value ko store krte h.

Python me **dict** **hash table** ka use karta hai **key-value pairs** store karne ke liye.

- **Main Purpose:** Searching, indexing, and storing data efficiently.
- **Fast Lookup:** Normal searching **O(n)** hoti hai, hashing **O(1)** time me search kar sakta hai.

```
["Amit", "Rahul", "Neha", "Ramesh"]
```

Agar **linear search** use karein toh **O(n) time** lagega.

```
h("Amit") = 3
h("Rahul") = 7
h("Neha") = 2
```

Lekin hashing ka **hash function** use karke:

Yeh values ek **hash table** me store hongy, jisme **direct O(1) access** possible hoga. And hash table ke liye we use dict in python

Python ke **dict** me **ek fixed-size array hota hai**, jisme **each index ek bucket** hoti hai. Har bucket me ek **key-value pair** store hota hai.

```
dict_data = {
    "name": "Abhishek",
    "age": 26,
    "city": "Delhi"
}
```

👉 Internally, Python ek **hash function** use karta hai jo keys ko ek **unique memory index** pe store karta hai.

Index	Hash(Key)	Key	Value
0	hash("age")	"age"	26
1	hash("city")	"city"	"Delhi"
2	hash("name")	"name"	"Abhishek"
3	...	...	...

Hashing ka use kya h?

✓ Hashing ka use:

1. **Data Structures:** HashMap, HashSet
2. **Cryptography:** Password hashing
3. **Databases:** Indexing
4. **Caching:** LRU Cache

Hash Table and Hash Function kya hota h ?

Hash Table wo data structure hota h jo ki key value ko store krta h.

Hash function ek **mathematical function** hai jo **input ko fixed size output** me convert karta hai.

$h(\text{key}) \rightarrow \text{hash value (index in table)}$

Ek **achha hash function**:

- **Deterministic hota hai:** Same input ke liye **same output** deta hai.
- **Uniformly distribute karta hai:** Data ko **proper spread** karta hai.
- **Fast compute hota hai:** $O(1)$ time me hashing kare

Ek simple hash function jo ek **number ko table ke index se map karega**:

```
def simple_hash(key, size):
    return key % size # Hashing by remainder

print(simple_hash(25, 10)) # Output: 5
print(simple_hash(35, 10)) # Output: 5 (Collision!)
```



Note - hashing ka use password security me kaise hote h ?

Normally hashing key par apply hoti hai jab hum hashmaps ya dictionaries use karte hain, kyunki hume fast lookup chahiye hota hai. Lekin password hashing ek different use case hai! Yahaan hashing ka kam data security ke liye hota hai, na ki fast retrieval ke liye.

Steps in Password Hashing

1. **User enters a password** → "mypassword123"

2. **Hash function apply hota hai** → Converts it into a **fixed-length hash**
3. **Hashed password store hota hai** → Instead of storing the plain text password
4. **Login ke waqt**, user ka entered password hash kiya jata hai & **compare hota hai stored hash se**

```
import hashlib

def hash_password(password: str) → str:
    return hashlib.sha256(password.encode()).hexdigest()

# Example password
password = "mypassword123"
hashed_password = hash_password(password)
print("Stored Hashed Password:", hashed_password)
```

🔑 Output (Hashed Password Example):

```
c157a79031e1c40f85931829bc5fc552...
```

Agar **hashing apply karein**:

Username	Hashed Password
abhishek	c157a79031e1c40f85931829bc5fc552...
user1	5e884898da28047151d0e56f8dc6292...

User jab login karega, toh **entered password ka hash generate** karke compare karna padega **stored hash se**.

Collisions in Hashing and Uski Techniques

Collision tab hota hai jab **do alag-alag keys** ka **same hash value aajaye**. Mtlb multiple key ko hash function me pass krne pe same index mile sote krne ke liye.

Example :

```
h(25) = 5
h(35) = 5 # Collision!
```

✅ Collision ko handle karne ke 3 main tareeke hain:

1. **Direct Address Table** : **Jab keys small aur limited range me hoti hain** tab hum ek simple array use kar sakte hain.
2. **Chaining (Separate Chaining)**
3. **Open Addressing (Linear Probing, Quadratic Probing, Double Hashing)**

1 Chaining (Separate Chaining)

- **Concept**: Hash table ke har index pe ek **linked list** hoti hai.
- **Jab collision hota hai toh** naye element ko us index ki linked list me daal dete hain.

Python Code

```

class HashTable:
    def __init__(self, size):
        self.size = size # size should be a nearest prime number less than actual size
        self.table = [[] for _ in range(size)] # Initialize with empty lists

    # Hash function to find index
    def hash_function(self, key):
        return key % self.size

    # Insert key-value pair
    def insert(self, key, value):
        index = self.hash_function(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value) # Update value
                return
        self.table[index].append((key, value)) # No collision or new key

    # Get value by key
    def get(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None # If key doesn't exist

    # Remove a key-value pair
    def remove(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the element
                return True
        return False # If key doesn't exist

    # Resize the hash table (doubling the size)
    def resize(self):
        self.size *= 2 # Double the size
        new_table = [[] for _ in range(self.size)] # New empty table
        for bucket in self.table:
            for k, v in bucket:
                new_index = self.hash_function(k) # Rehash old entries
                new_table[new_index].append((k, v))
        self.table = new_table # Update table to the new size

    # Display the hash table (for debugging)
    def display(self):
        for i, bucket in enumerate(self.table):

```



```

        if bucket:
            print(f"Index {i}: {bucket}")

# Example Usage:
ht = HashTable(10)
ht.insert(5, "Amit")
ht.insert(15, "Rahul") # Collision occurs, stored in linked list
ht.insert(25, "Priya") # Collision with 5, stored in the same bucket
ht.insert(35, "John") # Collision with 15, stored in the same bucket

print(ht.get(5)) # Output: Amit
print(ht.get(15)) # Output: Rahul
print(ht.get(25)) # Output: Priya

ht.remove(15) # Remove key 15
print(ht.get(15)) # Output: None (since 15 is removed)

# Resize the hash table
ht.resize()
ht.display()

```

✅ **Pros:** Simple & effective

❌ **Cons:** Extra space lagta hai linked list ke liye

2 Open Addressing

- **Concept:** Jab collision hota hai toh **next available space** find karte hain.
- **Methods:**
 1. **Linear Probing** (next slot check)
 2. **Quadratic Probing** (gap increasing)
 3. **Double Hashing** (new hash function)

1. Linear Probing (Next Slot Check)

Explanation :

Linear probing ek collision resolution technique hai jisme agar koi collision hoti hai (matlab jo index hum calculate karte hain, woh pehle se filled hai), toh hum next slot check karte hain. Agar woh bhi filled hai, toh next slot ko check karte hain, aur aise hi continue karte hain jab tak humein empty slot nahi mil jaata.

Yeh technique **primary clustering** ka reason ban sakti hai, jisme consecutive slots bhar jaate hain aur isse search, insertion aur deletion slow ho jaate hain.

Steps :

1. Key ka hash value calculate karo.
2. Agar index empty hai toh key ko uss slot mein insert karo.
3. Agar index occupied hai, toh next slot ko check karo (index + 1), aur aise hi check karte raho jab tak empty slot nahi mil jaata.

Code:

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        index = self._hash(key)

        # Linear probing for collision resolution
        while self.table[index] is not None:
            index = (index + 1) % self.size # Next slot check

        self.table[index] = (key, value)

    def get(self, key):
        index = self._hash(key)

        # Linear probing for searching the key
        while self.table[index] is not None:
            if self.table[index][0] == key:
                return self.table[index][1]
            index = (index + 1) % self.size # Next slot check

        return None

# Example usage
ht = HashTable(10)
ht.insert(5, "Amit")
ht.insert(15, "Rahul") # Collision, stored in next slot
print(ht.get(5)) # Output: Amit
print(ht.get(15)) # Output: Rahul
```

Time Complexity :

- **Average:** $O(1)$ (agar collision kam hain)
- **Worst-case:** $O(n)$ (jab table almost full ho ya primary clustering (Primary clustering occurs when multiple keys hash to the same index (or nearby indices), forming long clusters of occupied slots in the hash table. This makes future insertions slower because new elements must be placed farther away)ho)

Space Complexity: $O(n)$ (n elements store karne ke liye)

In-place: Haan (directly table ko update karta hai)

2. Quadratic Probing (Gap Increasing)

Explanation :

Quadratic probing ek improvement hai linear probing ka. Linear probing ki jagah quadratic probing mein hum quadratic sequence mein slots ko check karte hain (jaise $\text{index} + 1^2$, $\text{index} + 2^2$, $\text{index} + 3^2$, etc.). Isse **primary clustering** kaafi reduce ho jaata hai lekin **secondary clustering** (mtlb clustering to hoga but continuous ni hoga alg location pe hoga kyuki sare key same pattern me honge)ab bhi ho sakti hai.

Steps:

1. Key ka hash value calculate karo.
2. Agar index occupied hai, toh next slot $\text{index} + 1^2$, phir $\text{index} + 2^2$, aur aise hi check karte jao jab tak empty slot nahi mil jaata.

Code:

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        index = self._hash(key)
        i = 1

        # Quadratic probing for collision resolution
        while self.table[index] is not None:
            index = (index + i**2) % self.size # Quadratic gap
            i += 1

        self.table[index] = (key, value)

    def get(self, key):
        index = self._hash(key)
        i = 1

        # Quadratic probing for searching the key
        while self.table[index] is not None:
            if self.table[index][0] == key:
                return self.table[index][1]
            index = (index + i**2) % self.size # Quadratic gap
            i += 1

        return None

# Example usage
ht = HashTable(10)
ht.insert(5, "Amit")
ht.insert(15, "Rahul") # Collision resolved using quadratic probing
```

```
print(ht.get(5)) # Output: Amit
print(ht.get(15)) # Output: Rahul
```

Time Complexity :

- **Average:** $O(1)$ (agar collision kam hain)
- **Worst-case:** $O(n)$ (jab table full ho jaaye ya quadratic probing sabhi slots check kar le)

Space Complexity: $O(n)$ (n elements store karne ke liye)

In-place: Haan (directly table ko update karta hai)

3. Double Hashing (New Hash Function)

Explanation :

Double hashing mein hum do hash functions use karte hain. Jab collision hoti hai, toh hum ek second hash function use karte hain step size calculate karne ke liye. Yeh technique **primary** aur **secondary clustering** dono ko reduce karti hai, aur yeh hash table ko efficient banaata hai.

Steps :

1. Key ka hash value compute karo pehle hash function se.
2. Agar slot occupied hai, toh doosre hash function se step size calculate karo.
3. Table ko yeh step size use karke probe karo aur repeat karo jab tak empty slot nahi mil jaata.

Code:

```
class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size

    def _hash1(self, key):
        return key % self.size

    def _hash2(self, key):
        return 7 - (key % 7) # Secondary hash function

    def insert(self, key, value):
        index = self._hash1(key)
        step_size = self._hash2(key)

        # Double hashing for collision resolution
        while self.table[index] is not None:
            index = (index + step_size) % self.size

        self.table[index] = (key, value)

    def get(self, key):
        index = self._hash1(key)
        step_size = self._hash2(key)
```

```

# Double hashing for searching the key
while self.table[index] is not None:
    if self.table[index][0] == key:
        return self.table[index][1]
    index = (index + step_size) % self.size

return None

# Example usage
ht = HashTable(10)
ht.insert(5, "Amit")
ht.insert(15, "Rahul") # Double hashing for collision resolution
print(ht.get(5)) # Output: Amit
print(ht.get(15)) # Output: Rahul

```

Time Complexity :

- **Average:** $O(1)$ (agar collision kam hain)
- **Worst-case:** $O(n)$ (agar table full ho jaaye)

Space Complexity: $O(n)$ (n elements store karne ke liye)

In-place: Haan (directly table ko update karta hai)

Collision Resolution Techniques ki Comparison:

Method	Pros	Cons	Time Complexity (Average)	Time Complexity (Worst-case)
Linear Probing	Simple, easy to implement	Primary clustering ho sakti hai	$O(1)$	$O(n)$
Quadratic Probing	Primary clustering reduce ho jaati hai	Secondary clustering ho sakti hai	$O(1)$	$O(n)$
Double Hashing	Sabse efficient (primary & secondary clustering dono reduce hote hain)	Do hash functions required hote hain	$O(1)$	$O(n)$

Kab kaunsa use karein:

1. **Linear Probing:** Jab simplicity zyada important ho aur table mein thoda space ho.
2. **Quadratic Probing:** Jab primary clustering ko reduce karna ho, par secondary clustering thoda acceptable ho.
3. **Double Hashing:** Jab performance ko optimize karna ho aur primary aur secondary clustering dono minimize karne ho.

Load Factor and Rehashing Kya hota h ?

Load Factor (LF)

Load Factor ek important concept hai hashing ke implementation mein. Yeh ek metric hai jo batata hai ki hash table kitna "full" hai, yaani hash table mein kitni entries (elements) store ki gayi hain.

Formula:

$$\text{LoadFactor}(LF) = \text{Number of Elements in HashTable} / \text{Total Number of Slots (Capacity) in HashTable}$$

- Agar **LF = 0.5**, iska matlab hai ki hash table ka half space fill ho chuka hai.
- Agar **LF = 1**, iska matlab hai hash table poora bhar gaya hai (table ka size aur capacity equal hai).
- Agar **LF > 1**, iska matlab hai hash table ka size full ho chuka hai aur ab collision probability high ho sakti hai.

Load Factor ka role:

- Agar load factor high ho, toh **collisions** zyada hone lagte hain aur **performance degrade** ho sakti hai.
- Agar load factor low ho, toh hash table ki memory waste ho sakti hai, lekin performance achi hoti hai.

Rehashing

Rehashing ka concept tab aata hai jab load factor ek certain threshold ko cross kar leta hai. Isme hash table ka size increase kar diya jaata hai aur existing elements ko nayi table mein insert kiya jaata hai.

- **Threshold Load Factor:** Jab load factor ek certain value ko cross kar jaata hai (usually 0.7 ya 70%), tab rehashing kiya jaata hai.
- **Rehashing Process:** Nayi table ko create kiya jaata hai, jo pehle se bada hota hai, aur purane elements ko naye table mein phir se insert kiya jaata hai.
- **Rehashing ka purpose:** Yeh **collisions** ko reduce karne aur **search** aur **insert** operations ki performance ko improve karne ke liye hota hai.

Rehashing ka process:

1. Pehle hash table ka size double kiya jaata hai.
2. Phir purane elements ko nayi table mein **rehash** (re-insert) kiya jaata hai. Nayi table ke liye hash function ko dobara apply kiya jaata hai, kyunki table ka size badal gaya hota hai.

Code Example:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size # Initial size of the table
        self.table = [None] * self.size # Empty table
        self.num_elements = 0 # Number of elements currently in the table

    def _hash(self, key):
        return key % self.size

    def insert(self, key, value):
        # Check if load factor exceeds 0.7, trigger rehashing
        load_factor = self.num_elements / self.size
        if load_factor > 0.7:
            self._rehash()

        index = self._hash(key)

        # Linear probing for collision resolution
        while self.table[index] is not None:
            index = (index + 1) % self.size
```

```

self.table[index] = (key, value)
self.num_elements += 1

def get(self, key):
    index = self._hash(key)

    # Linear probing for searching the key
    while self.table[index] is not None:
        if self.table[index][0] == key:
            return self.table[index][1]
        index = (index + 1) % self.size

    return None

def _rehash(self):
    print("Rehashing...")
    # Old size and table
    old_table = self.table
    old_size = self.size

    # Double the size of the table
    self.size *= 2
    self.table = [None] * self.size
    self.num_elements = 0 # Reset number of elements for rehashing

    # Rehash all the old elements and insert them into the new table
    for item in old_table:
        if item is not None:
            self.insert(item[0], item[1])

# Example usage
ht = HashTable()
ht.insert(1, "Amit")
ht.insert(2, "Rahul")
ht.insert(3, "Raj")
ht.insert(4, "Neha")
ht.insert(5, "Vikram")
ht.insert(6, "Priya")
ht.insert(7, "Ravi") # Load factor will exceed 0.7, trigger rehashing
print(ht.get(5)) # Output: Vikram
print(ht.get(7)) # Output: Ravi

```

Steps in Rehashing :

1. **Check Load Factor:** Jab load factor threshold (usually 0.7) cross karta hai, toh rehashing ko trigger kiya jaata hai.
2. **Increase Table Size:** Naye table ka size double karna hota hai.
3. **Rehash Elements:** Purane elements ko nayi table mein insert karna hota hai.

4. **Recompute Hash:** New size ke according hash values ko dobara compute kiya jaata hai, taki naye indices calculate ho sakein.

Time Complexity :

- **Insert Operation:**
 - **Average-case:** $O(1)$
 - **Worst-case:** $O(n)$ (jab rehashing hota hai)
- **Rehashing:**
 - **Time Complexity:** $O(n)$ (kyunki purane elements ko naye table mein rehash karna padta hai)

Space Complexity :

- **Overall:** $O(n)$ (new table ke liye extra space ki zarurat hoti hai)

In-place: Nahi, rehashing mein naya table create hota hai, toh yeh in-place operation nahi hai.

Python: dict , set , defaultdict , Counter

Python me kuch important collections hain jo hashing par based hote hain, jinke through hum data ko efficiently store aur access kar sakte hain. Yeh collections `dict` , `set` , `defaultdict` , aur `Counter` hain. Inko samajhna aur use karna interview aur DSA mein kaafi helpful hota hai.

1. dict (Dictionary)

Python ka `dict` ek **key-value** store hai, jisme har key unique hoti hai aur har key ko ek value map ki jaati hai. Yeh hashing ka use karta hai, jisme keys ko hash karke unke corresponding value ko store kiya jaata hai.

Example:

```
# Creating a dictionary
person = {"name": "Amit", "age": 25, "city": "Delhi"}

# Accessing a value using the key
print(person["name"]) # Output: Amit

# Adding a new key-value pair
person["job"] = "Engineer"
print(person) # Output: {'name': 'Amit', 'age': 25, 'city': 'Delhi', 'job': 'Engineer'}

# Updating an existing value
person["age"] = 26
print(person["age"]) # Output: 26

# Deleting a key-value pair
del person["city"]
print(person) # Output: {'name': 'Amit', 'age': 26, 'job': 'Engineer'}
```

Key Points :

- **Time Complexity:** $O(1)$ for average case lookups, insertions, and deletions.
- **Space Complexity:** $O(n)$, where n is the number of key-value pairs.
- **In-place:** Yes, `dict` works in-place.

2. `set` (Set)

Python ka `set` ek collection hota hai jo **unique elements** store karta hai. Yeh **unordered** hota hai aur duplicate values ko allow nahi karta. Yeh bhi hashing ka use karta hai internally.

```
# Creating a set
numbers = {1, 2, 3, 4, 5}

# Adding elements
numbers.add(6)
print(numbers) # Output: {1, 2, 3, 4, 5, 6}

# Removing an element
numbers.remove(4)
print(numbers) # Output: {1, 2, 3, 5, 6}

# Checking membership
print(3 in numbers) # Output: True
print(7 in numbers) # Output: False

# Union of two sets
set2 = {4, 5, 6, 7}
union_set = numbers.union(set2)
print(union_set) # Output: {1, 2, 3, 5, 6, 4, 7}

# Intersection of two sets
intersection_set = numbers.intersection(set2)
print(intersection_set) # Output: {5, 6}
```

Key Points :

- **Time Complexity:** $O(1)$ for add, remove, and membership tests.
- **Space Complexity:** $O(n)$, where n is the number of elements.
- **In-place:** Yes, sets work in-place.

3. `defaultdict`

`defaultdict` ek special type ka dictionary hai jo default values assign karta hai jab hum kisi key ko access karte hain jo exist nahi karti. Isme ek default factory function hota hai jo new values ko generate karta hai.

Example:

```
from collections import defaultdict
```

```
# Creating a defaultdict with a default value of list
my_dict = defaultdict(list)

# Adding values to keys (list of values)
my_dict["fruits"].append("apple")
my_dict["fruits"].append("banana")
my_dict["veggies"].append("carrot")

print(my_dict) # Output: defaultdict(<class 'list'>, {'fruits': ['apple', 'banana'], 'veggies': ['carrot']})

# Creating a defaultdict with a default value of int
count_dict = defaultdict(int)

# Counting occurrences of words
words = ["apple", "banana", "apple", "cherry", "banana", "banana"]
for word in words:
    count_dict[word] += 1

print(count_dict) # Output: defaultdict(<class 'int'>, {'apple': 2, 'banana': 3, 'cherry': 1})
```

Key Points :

- **Time Complexity:** O(1) for access, insertion, and updates.
- **Space Complexity:** O(n), where n is the number of elements.
- **In-place:** Yes, `defaultdict` works in-place.
- **Use case:** Jab humein kisi key ki absence par default value chahiye hoti hai.

4. Counter

`Counter` ek special type ka dictionary hai jo **element frequencies** ko store karta hai. Yeh collection type **hashing** ka use karta hai aur kisi bhi sequence ke elements ko count karne ka easy way provide karta hai.

Example:

```
from collections import Counter

# Counting frequencies of characters in a string
text = "abracadabra"
counter = Counter(text)

print(counter) # Output: Counter({'a': 5, 'r': 2, 'b': 2, 'c': 1, 'd': 1})

# Finding the most common elements
print(counter.most_common(2)) # Output: [('a', 5), ('r', 2)]

# Updating the counter with new data
counter.update("newdata")
print(counter) # Output: Counter({'a': 6, 'r': 2, 'b': 2, 'n': 2, 'e': 2, 'd': 2, 'c': 2, 't': 1, 'w': 1})
```

Key Points :

- **Time Complexity:** $O(n)$ for counting elements where n is the number of elements in the iterable.
- **Space Complexity:** $O(n)$, where n is the number of unique elements.
- **In-place:** Yes, `Counter` works in-place.
- **Use case:** Jab humein element frequency count karni ho.

Summary:

1. `dict` (**Dictionary**): Hash-based, stores key-value pairs, used for mapping data.
2. `set` (**Set**): Hash-based, stores unique elements, no duplicates.
3. `defaultdict` : A type of dictionary with a default value for non-existing keys.
4. `Counter` : A special dictionary for counting frequencies of elements.

Agr Hashing ko scratch se implement krna ho to ?

Normally hum tuple concept use krte h jo ki hmne chaining me use kiya h . Jaha ek list h jisme bahut sare list h ek bucket ke taur pe aur usme har key value ko tuple ke form m store kr rhe h. But agr hame linked list ke taur pe implement krna hua to we can do like this -

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None # Next pointer for the linked list

class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * size # Initialize table with None (empty)

    def _hash(self, key):
        return key % self.size # Hash function

    def insert(self, key, value):
        index = self._hash(key)

        # Create a new node
        new_node = Node(key, value)

        # Check if the slot is empty
        if not self.table[index]:
            self.table[index] = new_node
        else:
            # Collision: Add new node at the beginning of the linked list
            new_node.next = self.table[index]
            self.table[index] = new_node
```

```

def get(self, key):
    index = self._hash(key)
    current = self.table[index]

    # Search in the linked list at the hashed index
    while current:
        if current.key == key:
            return current.value
        current = current.next
    return None

def delete(self, key):
    index = self._hash(key)
    current = self.table[index]
    prev = None

    # Search for the key in the linked list and remove it
    while current:
        if current.key == key:
            if prev: # Not the first element
                prev.next = current.next
            else: # First element
                self.table[index] = current.next
            return True
        prev = current
        current = current.next
    return False

def display(self):
    for i, node in enumerate(self.table):
        print(f"Index {i}: ", end="")
        current = node
        while current:
            print(f"({current.key}, {current.value}) →", end=" ")
            current = current.next
        print("None")

# Example Usage:
ht = HashTable(10)
ht.insert(5, "Amit")
ht.insert(15, "Rahul") # Collision, will be added as a new node in the linked list
ht.insert(25, "Mohan") # Another collision, same index

# Retrieve values by key
print(ht.get(5)) # Output: Amit
print(ht.get(15)) # Output: Rahul
print(ht.get(25)) # Output: Mohan

# Delete a key-value pair

```

```
ht.delete(15)
print(ht.get(15)) # Output: None (since it was deleted)

# Display the hash table structure
ht.display()
```

Explanation of the Linked List Approach:

1. Node Class:

- `Node` class ek **linked list node** ko represent karta hai. Har node me `key`, `value`, aur `next` pointer hota hai. `next` pointer next node ko point karta hai (agar koi collision hota hai).

2. Insert Operation:

- Jab koi new key-value pair insert karna hota hai, toh sabse pehle hum hash index calculate karte hain. Agar index pe koi value already stored hai, toh hum us bucket me **linked list** me new node insert karte hain.
- Hum new node ko bucket ke **head** me insert karte hain, isliye pehle insert hone wale item ko last me jaana padta hai.

3. Get Operation:

- Jab key ko get karte hain, toh hum hashed index pe linked list me search karte hain. Agar key milti hai toh corresponding value return karte hain.

4. Delete Operation:

- Jab key ko delete karte hain, toh hum linked list me node ko search karte hain. Agar wo node milti hai toh usse delete karte hain (agar wo first node hai toh head ko update karte hain).

Difference Between Tuple and Linked List:

- **Tuple:** Static storage structure, easy to use for small scale operations, **faster** for simple scenarios, but doesn't have the flexibility of linked lists.
- **Linked List:** Dynamic structure, offers more flexibility, allows efficient **insertions** and **deletions** at any point, but requires managing pointers and is more memory intensive.